Absolutely. I'll base the notes on the uploaded **"Differentiation"** chapter and optimize them for **interview preparation using the 80/20 principle**—focusing on what you are most likely to need for ML/AI engineering rather than reproducing every proof.

The source explicitly frames differentiation as important for machine learning because it enables **optimization and gradient descent**.

Differentiation — 80/20 Interview Notes

1. The Big Picture

What is differentiation?

Differentiation measures the instantaneous rate of change of a function.

For a function:

$$y = f(x)$$

its derivative at $x$is:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

In simple terms:

Derivative = how much the output changes when the input changes by a very small amount.

The source also gives the geometric interpretation:

Derivative = slope of the tangent line.

And in physics:

Derivative of position with respect to time = velocity.

These interpretations are directly developed in the chapter.

---

2. Why Does Differentiation Matter in Machine Learning?

This is probably the **most important interview connection**.

Machine learning usually involves:

Model → Prediction → Loss → Optimization

We want to find parameters that minimize the loss.

For example:

$$L(w) = (w-3)^2$$

We want:

$$\min_w L(w)$$

The derivative tells us the direction in which the loss is increasing/decreasing.

$$L'(w) = 2(w-3)$$

Then gradient descent uses the derivative:

$$w_{\text{new}} = w - \eta L'(w)$$

where:

- w= parameter
- η= learning rate
- $L'(w)$= gradient

ML connection

Training

|

▼

Model prediction

|

▼

Loss L

|

▼

Differentiation

|

▼

Gradient

|

▼

Gradient Descent

|

▼

Updated model parameters

The chapter explicitly identifies differentiation as a key component of optimization and connects derivatives with gradient descent.

---

3. Average Rate of Change

Before understanding the derivative, understand **average rate of change**.

Given:

$$f(x)$$

between $x_1$and x_2:

$$\frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

This is the slope of the **secant line**.

Example

Suppose:

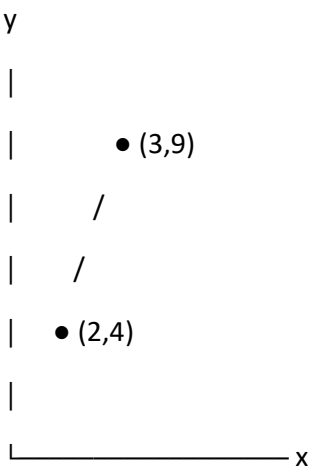
$$f(x) = x^2$$

Between $x = 2$and $x = 3$:

$$\frac{9 - 4}{3 - 2} = 5$$

So the average rate of change is:

Diagram



secant

The chapter introduces differentiation through exactly this idea of average velocity and the slope between two points.

---

4. Derivative

The derivative is what happens when the two points become infinitely close.

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Intuition

Two points far apart

secant

Points getting closer

/

●

tangent

At the limit:

/

/

●

/

So:

Derivative = instantaneous slope

The formal definition in the source is the limit of the difference quotient.

---

5. Most Important Example: x^2

Let:

$$f(x) = x^2$$

Derivative:

$$f'(x) = 2x$$

At:

$$x = 3$$

the derivative is:

$$f'(3) = 6$$

Meaning:

Around $x = 3$, a small increase of 1 in $x$produces approximately a 6-unit increase in $f(x)$.

The source derives $f'(x) = 2x$directly from the definition.

---

6. Differentiability

Definition

A function is **differentiable at** $x_0$if:

$$\lim_{x \to x_0} \frac{f(x) - f(x_0)}{x - x_0}$$

exists.

That limit is the derivative:

$$f'(x_0)$$

Interview answer

If asked:

"What does differentiability mean intuitively?"

Say:

A function is differentiable at a point when it has a well-defined local slope there. Geometrically, a tangent exists at that point.

---

7. Differentiability vs Continuity

This is a common conceptual interview question.

Important relationship

$$\boxed{\text{Differentiable} \Rightarrow \text{Continuous}}$$

But:

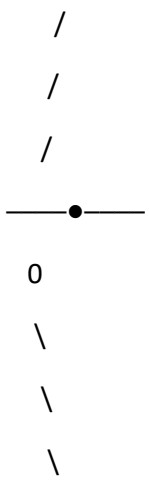
$$\boxed{\text{Continuous} \not\Rightarrow \text{Differentiable}}$$

The chapter explicitly states this theorem.

Example

$$f(x) = | x |$$

Graph:



It is continuous at 0.

But it has a sharp corner.

Therefore:

$$f'(0)$$

does not exist.

The source uses $| x |$ at $x = 0$ as its main example of a continuous but non-differentiable function.

Interview question

Q: Is every continuous function differentiable?

Answer:

No. Differentiability implies continuity, but continuity does not imply differentiability. $f(x) = | x |$ at $x = 0$ is a simple counterexample.

---

8. Local Linear Approximation

This is **very important for ML**.

A differentiable function can locally be approximated by a straight line:

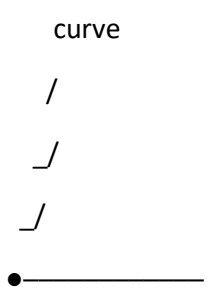
$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

More formally, the source expresses this as:

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + o(|\ x - x_0\ |)$$

Intuition

A complicated curve:



When you zoom in around a point, the curve looks increasingly like a line.

Why ML cares

This idea becomes extremely important in optimization.

Near the current parameter value:

$$L(w + \Delta w) \approx L(w) + \nabla L(w)^T \Delta w$$

This tells us how changing parameters affects the loss locally.

---

9. Differentiation Rules — HIGH PRIORITY

These are the **80/20 formulas you should memorize**.

Constant multiple

$$(cf)' = cf'$$

Example:

$$f(x) = 5x^2$$
$$f'(x) = 10x$$

Sum rule

$$(f+g)' = f' + g'$$

Example:

$$f(x) = x^2 + x^3$$
$$f'(x) = 2x + 3x^2$$

Product rule

$$(fg)' = f'g + fg'$$

Example:

$$f(x) = x^2 \sin x$$
$$f'(x) = 2x \sin x + x^2 \cos x$$

The source lists these rules explicitly.

---

10. Chain Rule — ⭐ ⭐ ⭐ VERY IMPORTANT

If:

$$y = f\big(g(x)\big)$$

then:

$$\boxed{\frac{dy}{dx} = f'\big(g(x)\big)g'(x)}$$

This is the **chain rule**.

The source emphasizes that composition is particularly important for neural networks because each neural-network layer can be viewed as a function composed with other functions.

Simple example

Let:

$$y = (3x + 2)^2$$

Break it into:

$$u = 3x + 2$$

and:

$$y = u^2$$

Then:

$$\frac{dy}{du} = 2u$$

and:

$$\frac{du}{dx} = 3$$

Therefore:

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$
$$= 2u(3)$$
$$= 6(3x + 2)$$
$$= 18x + 12$$

Diagram

x

|

▼

u = 3x + 2

|

▼

$y = u^2$

Backward:

dy/dy = 1

|

▼

dy/du = 2u

|

×

du/dx = 3

|

▼

dy/dx = 6u

This is essentially the fundamental idea behind **backpropagation**.

---

11. Chain Rule → Neural Networks

This is one of the **highest-value interview connections**.

Suppose:

$$x \to z_1 \to z_2 \to L$$

where:

$$z_1 = f_1(x)$$
$$z_2 = f_2(z_1)$$
$$L = f_3(z_2)$$

Then:

$$\frac{dL}{dx} = \frac{dL}{dz_2}\frac{dz_2}{dz_1}\frac{dz_1}{dx}$$

Neural-network picture

Forward pass

——————————>

x → Layer 1 → Layer 2 → Layer 3 → Loss

<———————

Backpropagation

Forward pass calculates values.

Backward pass calculates derivatives/gradients.

The source explicitly says neural networks are huge composed functions and their derivatives are calculated by repeated application of the chain rule.

---

12. Elementary Derivatives — MEMORIZE

These are the important ones from the chapter:

$$\frac{d}{dx}x^n = nx^{n-1}$$
$$\frac{d}{dx}\sin x = \cos x$$
$$\frac{d}{dx}\cos x = -\sin x$$
$$\frac{d}{dx}e^x = e^x$$
$$\frac{d}{dx}\log x = \frac{1}{x}$$

The source presents these as the core elementary derivatives to remember.

---

13. Sigmoid Derivative — ⭐ ⭐ ⭐ ML Interview

Sigmoid:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Its derivative simplifies beautifully:

$$\boxed{\sigma'(x) = \sigma(x)\big(1-\sigma(x)\big)}$$

Why this matters

Sigmoid was historically important in neural networks.

Its derivative becomes very small when the sigmoid saturates near 0 or 1.

That connects to the **vanishing-gradient problem**.

Example:

large negative x x ≈ 0 large positive x

sigmoid ≈ 0 ≈ 0.5 ≈ 1

gradient ≈ 0 ≈ 0.25 ≈ 0

So gradients can become very small.

---

14. ReLU — ⭐ ⭐ ⭐ ML Interview

ReLU:

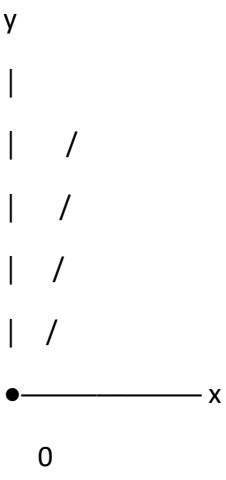
$$ReLU(x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

Derivative:

$$ReLU'(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

The source notes that ReLU is not differentiable at zero, but this is generally not a practical obstacle; a derivative value can be assigned at that point.

Diagram



Interview question

Q: Why can ReLU be used if it isn't differentiable at 0?

Answer:

ReLU is non-differentiable only at one point. In practical neural-network training, encountering exactly zero is not generally problematic, and an arbitrary subgradient such as 0 can be assigned there.

---

15. Higher-Order Derivatives

First derivative:

$$f'(x)$$

Second derivative:

$$f''(x)$$

Third derivative:

$$f'''(x)$$

and generally:

$$f^{(n)}(x)$$

The source notes that second derivatives contain useful information about minima and maxima.

Example

$$f(x) = x^2$$

First derivative:

$$f'(x) = 2x$$

Second derivative:

$$f''(x) = 2$$

The positive second derivative indicates upward curvature.

---

16. Taylor Expansion

A function can be approximated around $x_0$using derivatives:

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \cdots$$

The chapter introduces Taylor's theorem and Taylor expansions as polynomial approximations using higher derivatives.

80/20 understanding

You don't need to memorize every proof.

Remember:

Taylor expansion uses derivatives to approximate a complicated function locally with a polynomial.

---

17. Numerical Differentiation

Sometimes we don't know the analytical derivative.

Then we can approximate it numerically.

Forward difference

$$f'(x) \approx \frac{f(x + h) - f(x)}{h}$$

Example

Suppose:

$$f(x) = x^2$$

At $x = 3$, choose:

$$h = 0.001$$

Then:

$$f'(3) \approx \frac{3.001^2 - 3^2}{0.001}$$

Approximately:

$$6.001$$

True derivative:

$$6$$

---

18. Central Difference

Instead of only looking forward:

$$\frac{f(x+h) - f(x)}{h}$$

we can use:

$$\boxed{\frac{f(x+h) - f(x-h)}{2h}}$$

This is the **symmetric/central difference**.

The source presents forward, backward, and symmetric finite differences and notes that symmetric differences are generally better approximations.

---

19. Analytical vs Numerical Differentiation

Method	**Idea**	**Typical use**
Analytical	Derive exact formula	ML mathematics
Numerical	Approximate using function evaluations	Gradient checking/testing
Automatic differentiation	Compute derivatives systematically through operations	Modern ML frameworks

For your interview, understand the distinction.

Important

Don't confuse:

Numerical differentiation

with

Automatic differentiation (Autodiff).

Modern frameworks such as PyTorch primarily use automatic differentiation for computing gradients.

---

20. Implementation — From the Source

The chapter implements a function abstraction with a derivative method.

A simplified version:

```
import numpy as np



class Function:
    def __call__(self, x):
        raise NotImplementedError

    def prime(self, x):
        raise NotImplementedError



class Sigmoid(Function):

    def __call__(self, x):
        return 1 / (1 + np.exp(-x))

    def prime(self, x):
        y = self(x)
```

```
        return y * (1 - y)
```

Usage:

```
sigmoid = Sigmoid()


x = 2.0


print(sigmoid(x))
print(sigmoid.prime(x))
```

The important design idea is:

```
Function
  |
  ├── __call__()
  |     ↓
  |    f(x)
  |
  └── prime()
        ↓
       f'(x)
```

---

21. Composition Implementation

The source then builds a Composition abstraction to apply the chain rule.

A simplified implementation:

```
class Linear(Function):


   def __init__(self, a, b):
      self.a = a
      self.b = b
```

```
    def __call__(self, x):
        return self.a * x + self.b

    def prime(self, x):
        return self.a


class Composition(Function):

    def __init__(self, *functions):
        self.functions = functions

    def __call__(self, x):
        for f in self.functions:
            x = f(x)
        return x

    def prime(self, x):
        values = [x]

        for f in self.functions:
            x = f(x)
            values.append(x)

        derivative = 1

        for f, value in zip(self.functions, values[:-1]):
            derivative *= f.prime(value)
```

```
        return derivative
```

Example:

```
f = Composition(
    Linear(2, 0),
    Linear(3, 0),
    Linear(4, 0)
)


print(f.prime(5))
```

The derivative is:

$$2 \times 3 \times 4 = 24$$

The source uses this exact conceptual example and verifies that the composition has derivative 24.

---

22. The Most Important ML Connection

Everything we've learned can now be connected:

Function

  |

  ▼

Derivative

  |

  ▼

Gradient

  |

  ▼

Loss Function

  |

▼

Gradient Descent

|

▼

Parameter Update

|

▼

Training Neural Network

And inside the neural network:

Forward:

Input

↓

Layer 1

↓

Layer 2

↓

Layer 3

↓

Prediction

↓

Loss

Backward:

Loss

↓

∂L/∂Layer3

↓

∂L/∂Layer2

↓

∂L/∂Layer1

↓

Parameter gradients

↓

Update parameters

The mathematical engine underneath the backward pass is the **chain rule**.

---

23. Interview Example: One-Neuron Network

Suppose:

$$z = wx + b$$

and:

$$\hat{y} = z$$

Loss:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

We want:

$$\frac{\partial L}{\partial w}$$

Use chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w}$$

Now:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$
$$\frac{\partial \hat{y}}{\partial z} = 1$$
$$\frac{\partial z}{\partial w} = x$$

Therefore:

$$\boxed{\frac{\partial L}{\partial w} = (\hat{y} - y)x}$$

This is an excellent interview problem because it demonstrates:

- derivatives
- chain rule
- loss
- gradients
- neural networks
- backpropagation

---

24. Gradient vs Derivative

Derivative

Usually associated with a single-variable function:

$$f: \mathbb{R} \to \mathbb{R}$$
$$f'(x)$$

Gradient

For a scalar function with multiple variables:

$$f(x_1, x_2, \dots, x_n)$$

the gradient is:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

The source introduces this as the multivariable generalization of derivatives.

Example

$$f(x, y) = x^2 + 3y^2$$

Then:

$$\frac{\partial f}{\partial x} = 2x$$
$$\frac{\partial f}{\partial y} = 6y$$

Therefore:

$$\boxed{\nabla f = \begin{bmatrix} 2x \\ 6y \end{bmatrix}}$$

---

25. 80/20 Priority List

For your next **AI/ML interview**, I would divide this chapter like this:

Priority	Concept	Importance
🔴 P0	Meaning of derivative	Must know
🔴 P0	Difference quotient	Must know
🔴 P0	Derivative as slope/rate of change	Must know
🔴 P0	Basic derivative rules	Must know
🔴 P0	Chain rule	**Extremely important**
🔴 P0	Chain rule → backpropagation	**Extremely important**

Priority	Concept	Importance
🔴 P0	Gradient	**Extremely important**
🔴 P0	Differentiation → optimization	**Extremely important**
🔴 P0	Sigmoid derivative	Important
🔴 P0	ReLU derivative	Important
🟡 P1	Differentiability vs continuity	Important
🟡 P1	Local linear approximation	Important
🟡 P1	Higher-order derivatives	Useful
🟡 P1	Numerical differentiation	Useful
🟡 P1	Taylor expansion intuition	Useful
🟢 P2	Formal proofs	Low interview ROI
🟢 P2	Higher-order product-rule proof	Low interview ROI
🟢 P2	Weierstrass function	Very low interview ROI

This follows the source's emphasis while filtering it for your interview goal. The chapter itself concludes that differentiation, its rules, composition/chain rule, finite differences, backpropagation, and gradient descent are central to machine-learning training.

---

26. Interview Questions You Should Be Able to Answer

Basic

1. What is differentiation?

Differentiation measures the instantaneous rate of change of a function. Geometrically, the derivative is the slope of the tangent line.

2. What is the derivative of x^n?

$$nx^{n-1}$$

3. What is the derivative of e^x?

$$e^x$$

4. What is the derivative of $\log x$?

$$1/x$$

Conceptual

5. What is the difference between continuity and differentiability?

Differentiability implies continuity, but continuity does not imply differentiability.

6. Why is $|x|$ not differentiable at 0?

Because the left-hand and right-hand slopes are different, so there is no unique tangent slope.

7. What does the derivative tell us?

It tells us the local rate of change and therefore how the output responds to a small change in the input.

ML

8. Why do we need derivatives in machine learning?

We use derivatives to determine how the loss changes with respect to model parameters. These gradients allow optimization algorithms such as gradient descent to update the parameters.

9. What is the chain rule?

$$\big(f(g(x))\big)' = f'\big(g(x)\big)g'(x)$$

10. Why is the chain rule important in neural networks?

A neural network is a composition of functions. Backpropagation repeatedly applies the chain rule to calculate how the loss changes with respect to parameters in earlier layers.

11. What is a gradient?

The gradient is a vector containing the partial derivatives of a scalar function with respect to its variables.

12. Why does gradient descent subtract the gradient?

$$\theta_{\text{new}} = \theta - \eta \nabla L(\theta)$$

The gradient points in the direction of greatest local increase of the loss, so moving in the opposite direction tends to decrease the loss locally.

---

27. One-Minute Interview Explanation

If the interviewer says:

"Explain differentiation and its importance in ML."

A strong answer would be:

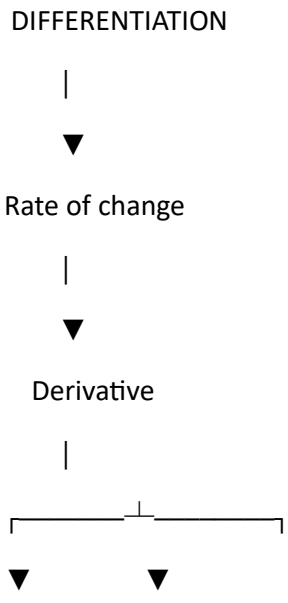
Differentiation measures the instantaneous rate of change of a function. The derivative can be understood geometrically as the slope of the tangent at a point. In machine learning, we typically have a loss function that depends on model parameters, and we need to find parameter values that minimize this loss. Derivatives tell us how the loss changes when each parameter changes. For multiple parameters, these partial derivatives form the gradient. Optimization algorithms such as gradient descent use this gradient to update the parameters. In neural networks, the model is a composition of many functions, so backpropagation uses the chain rule repeatedly to calculate gradients from the loss back through the layers.

That answer covers the **core 20% that gives you most of the interview value**.

---

28. Final Mental Model

Remember this:



Local slope Local approximation

|

▼

Gradient

|

▼

Loss w.r.t. parameters

|

▼

Chain Rule

|

▼

Backpropagation

|

▼

Gradient Descent

|

▼

Model Training

The 7 things I would memorize first

1. **Derivative = instantaneous rate of change**
2. **Derivative = tangent slope**
3. **Power rule:** $d(x^n)/dx = nx^{n-1}$
4. **Product rule**
5. **Chain rule**
6. **Gradient = vector of partial derivatives**
7. **Backpropagation = repeated chain rule**

The chapter's closing summary makes the same core connection: differentiation provides the rules for decomposing complex functions, the chain rule underpins backpropagation, and finite-difference ideas connect to gradient descent.

If you master those seven plus the **one-neuron backpropagation example**, you have the high-value foundation from this chapter for an ML interview.